

# Research Execution & Reading Roadmap

## Branchless SIMD Vector Acceleration of $\text{GF}(2^{128})$ Arithmetic

Parth Sinha

September 22, 2026

## 1 Project Overview & Strategy

This document serves as the operational tracker for the third paper in the systems research portfolio. The objective is to design, implement, and formally analyze a constant-time, branchless vector acceleration engine for  $\text{GF}(2^{128})$  finite field arithmetic targeting flagship IEEE venues (*IEEE Transactions on Computers* / *TCAS-I*).

### 1.1 Core Research Objectives

- **Mathematical Formulation:** Eliminate conditional long-division loops by constructing a static binary projection matrix  $\mathbf{M}_{red} \in \text{GF}(2)^{128 \times 127}$  for modular reduction.
- **Microarchitectural Mapping:** Map  $8 \times 8$  bit-matrix transformations onto wide SIMD vector registers using Galois Field New Instructions (`GF2P8AFFINEQB` / `VPCLMULQDQ`).
- **Side-Channel Resistance:** Prove complete data-independent execution timing using the `dudduct` statistical framework ( $|t| < 4.5$ ).

—

## 2 Targeted MIT 6.004 Lecture Notes Reading List

To build the microarchitectural foundation for the paper’s hardware hazard analysis, the following sections from the 663-page MIT 6.004 *Computation Structures* course notes are extracted for mandatory study. All other sections (compilers, OS, assembly) are explicitly out of scope.

### 2.1 Mandatory Reading Roadmap

### 2.2 Detailed Module Breakdown & Study Checklist

#### 2.2.1 1. L04: Combinational Logic (Pages 79–108)

- ☐ **Read:** Truth Tables, Boolean Algebra, Logic Gates, XOR Networks (pp. 79–95).
- ☐ **Read:** Propagation Delay ( $t_{PD}$ ) and Contamination Delay ( $t_{CD}$ ) (pp. 100–108).
- ☐ **Paper Note:** Extract formula proving that vector XOR additions have no carry-chain propagation delay, establishing  $t_{PD, \text{XOR}} = O(1)$ .

| Lecture Module                               | Page Range  | Core Concepts to Extract            | Paper Application                           |
|----------------------------------------------|-------------|-------------------------------------|---------------------------------------------|
| <b>L04: Combinational Logic</b>              | pp. 79–108  | Prop. Delay ( $t_{PD}$ ), XOR Trees | $O(1)$ delay in $GF(2^m)$ vs integer adders |
| <b>L05–L06: Sequential Logic &amp; FSMs</b>  | pp. 115–148 | Setup/Hold times, FSM Loops         | Multi-cycle sequential loop hazards         |
| <b>L07–L08: Performance &amp; Pipelining</b> | pp. 168–205 | Latency, Throughput, Bottlenecks    | Hardware execution cycle metrics            |
| <b>L15: Pipelining the Beta</b>              | pp. 430–472 | Hazards, Flushes, BTB Misses        | Control hazard & pipeline flush theory      |
| <b>L14: Memory Hierarchy (Skim)</b>          | pp. 382–400 | Cache line hit/miss penalties       | Side-channel cache timing vulnerabilities   |

Table 1: Targeted reading map for MIT 6.004 Lecture Notes.

### 2.2.2 2. L05 & L06: Sequential Logic & Finite State Machines (Pages 115–148)

- **Read:** Setup ( $t_S$ ) and Hold ( $t_H$ ) times, Register Timing Constraints (pp. 115–122).
- **Read:** FSM State Transitions and Next-State Logic (pp. 135–148).
- **Paper Note:** Use FSM transition equations to formally demonstrate why standard division loops impose sequential multi-cycle dependencies across 127 iterations.

### 2.2.3 3. L07 & L08: Performance Measures & Design Tradeoffs (Pages 168–205)

- **Read:** Latency vs. Throughput Metrics (pp. 168–175).
- **Read:** Pipelining Rules and Structural Bottlenecks (pp. 176–190).
- **Paper Note:** Define latency ( $T_{\text{exec}}$ ) and instruction throughput (GB/s) parameters for evaluation against OpenSSL/lib sodium baselines.

### 2.2.4 4. L15: Pipelining the Beta Processor (Pages 430–472)

- **Read:** Control Hazards, Data Hazards, and Branch Delay Slots (pp. 430–445).
- **Read:** Pipeline Flushes and Branch Target Buffer (BTB) Misses (pp. 446–460).
- **Paper Note:** Detail the exact mathematical penalty model for pipeline flushes ( $T_{\text{total}} = T_{\text{exec}} + N_{\text{miss}} \cdot \Delta t_{\text{flush}}$ ).

### 2.2.5 5. Explicit Non-Read List (Skip Completely)

- **Skip:** L01–L03 (Basics of Information, Digital Abstraction, CMOS Transistors)
- **Skip:** L10a–L12 (Assembly Language, Models of Computation, Compilers, Stacks)
- **Skip:** L13 (Building the Beta Processor)
- **Skip:** L16–L22 (Virtual Memory, OS Virtualization, Devices, Interrupts, Concurrency)

### 3 Execution Timeline & Milestone Tracker

| Phase / Week | Target Deliverable                                                                      | Status      |
|--------------|-----------------------------------------------------------------------------------------|-------------|
| Week 1–2     | Complete MIT 6.004 targeted readings; derive $\mathbf{M}_{red}$ matrix.                 | In Progress |
| Week 3–4     | Implement vector SIMD kernel / AVX-512 GF2P8AFFINEQB pipeline.                          | Pending     |
| Week 5–6     | Run <code>perf stat</code> cycle counts and <code>dudect</code> constant-time analysis. | Pending     |
| Week 7–8     | Finalize IEEE manuscript draft and upload preprint to Preprints.org.                    | Pending     |

Table 2: Project execution schedule.

### 4 Application Integration (RWTH Aachen File)

Upon completion, this third paper joins *Aparajita* (TKDE) and *vphilox* (TPDS) in your research portfolio. Together with a 170/170 GRE Quant score and 3 years of software engineering experience, this preprint serves as direct evidence for the Examination Board (*Prüfungsausschuss*) to issue a Conditional Admission (*mit Auflagen*).